LAPORAN PRAKTIKUM
Sistem Operasi

MODUL 06
(Sekuensial dan Konkuren)

Disusun Oleh:
Ryan Gabriel Togar Simamora (2311104045)
Kelas S1SE-07-02

Dosen : Yudha Islami Sulistya, S.Kom., M.Cs

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2025

BAB I
PENDAHULUAN

1.1 Dasar Teori

A. Sekuensial dan Konkuren Program

Secara dasar, program berjalan secara sekuensial, di mana instruksi dieksekusi satu per satu (statement per statement) tanpa jeda atau tumpang tindih. Namun, OS memperkenalkan konsep Program Konkuren (Concurrent Processing) untuk menjalankan banyak komputasi "pada saat yang bersamaan".

Konkurensi:

- Paralelisme sejati, terjadi pada komputer dengan banyak CPU (core), di mana tugas benar benar dieksekusi secara serentak.
- Ilusi Konkurensi, terjadi pada komputer 1 CPU melalui teknik multitasking/interleaving. OS bergantian (interleaving) mengeksekusi satu proses selama beberapa milidetik sebelum beralih ke proses lain. Pergantian cepat ini membuat semua proses terlihat berjalan bersamaan.

Kategori Multitasking:

- Timesharing, memberikan prioritas yang sama untuk setiap komputasi, memperbolehkan komputasi berjalan dan berakhir sesuai dengan durasi yang telah ditentukan.
- Realtime, memberikan prioritas yang berbeda untuk setiap komputasi karena terdapat performansi waktu yang ingin dicapai. Realtime akan memberikan prioritas dan menjadwalkan proses secara terencana.

Tujuan Praktikum:

1. Praktikan memahami perbedaan konsep sekuensial dan konkuren pada Xinu
2. Praktikan mampu membuat program konkuren pada Xinu

Manfaat Praktikum :

1. Memahami mekanisme eksekusi proses dalam sistem operasi
2. Meningkatkan keterampilan dalam pemrograman konkuren

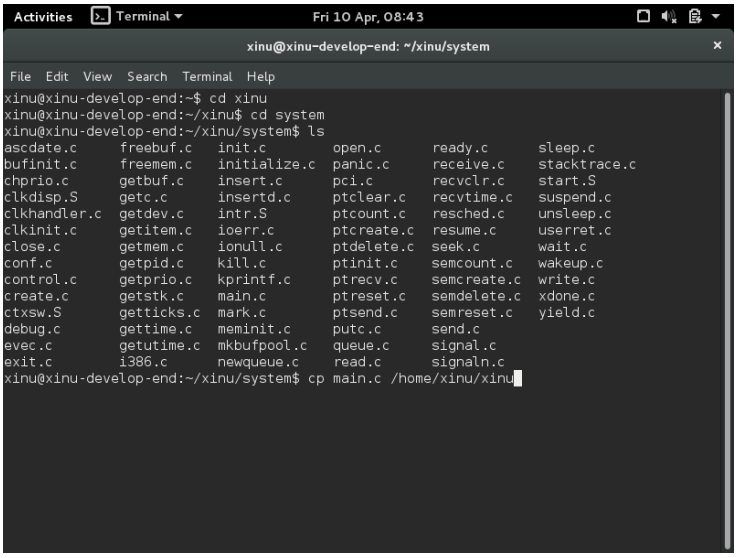
BAB II
HASIL PRAKTIKUM

2.1 Guided Praktikum

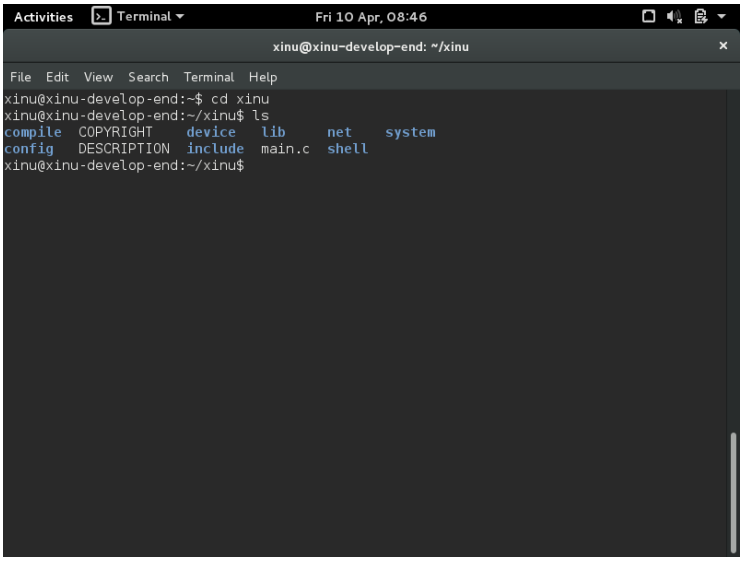
A. Sekuensial Program

Pada sekuensial program, prosesor akan mengeksekusi instruksi satu per satu. Perhatikan langkah membuat kode di main.c berikut ini:

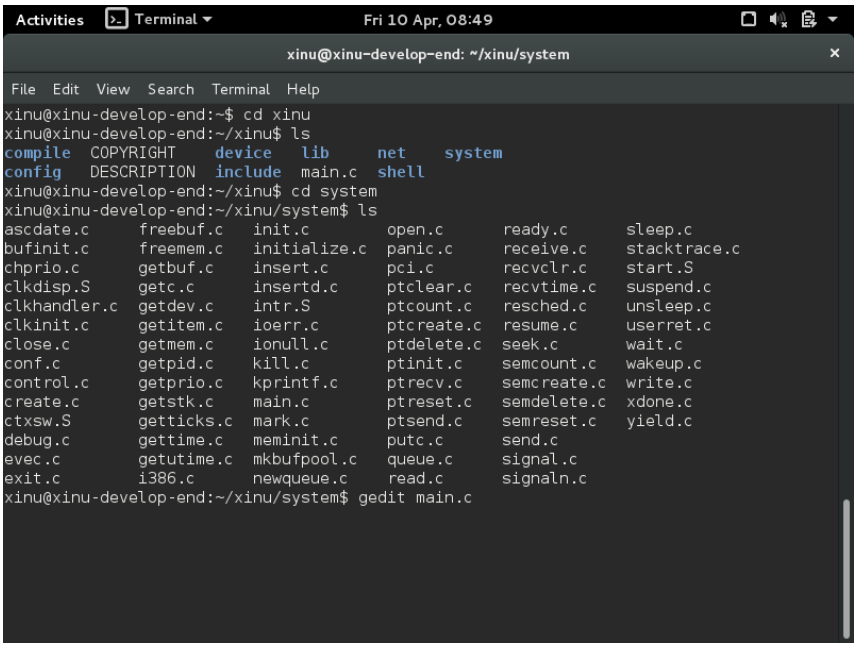
Pertama, copy main.c yang ada di direktori system ke direktori home/xinu/xinu.

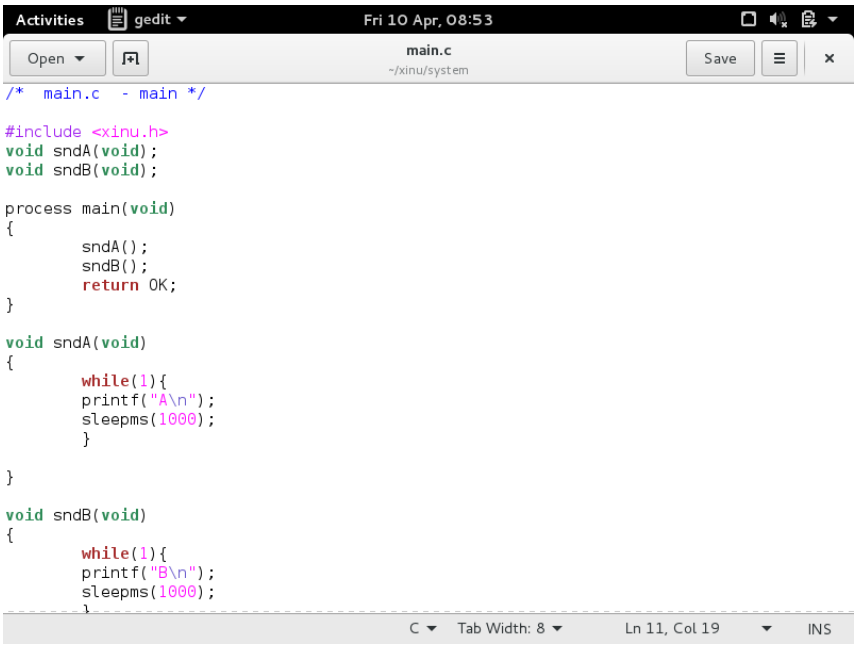


Jika berhasil, maka list dari direktori xinu akan seperti di bawah ini.



Selanjutnya, kita berikan perintah **gedit main.c** pada direktori system untuk membuka dan mengedit file, lalu edit isi file main.c seperti gambar di bawah (ini aku ikutin programnya dari modul praktikum). Jangan lupa untuk klik save.





Selanjutnya, kita kembali ke direktori xinu dengan memberikan perintah **cd ..**, lalu masuk ke direktori **compile**. Lalu, lakukan perintah **make clean** dan **make**. Jika berhasil, maka tampilannya akan seperti ini.

```
Activities  Terminal  Fri 10 Apr, 08:57
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
```

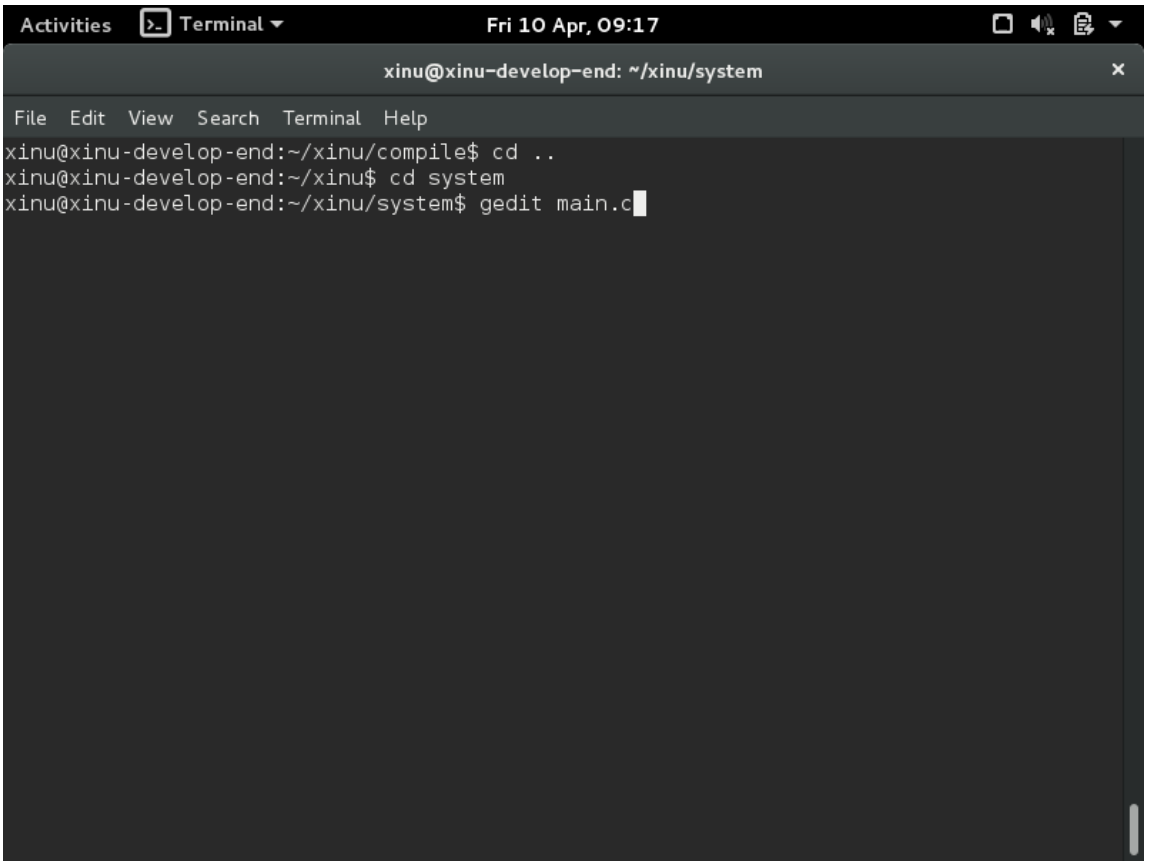
Setelah berhasil, berikan perintah sudo minicom, lalu masukkan password. Setelah itu, start **backend** pada Virtualbox.
Maka hasilnya akan seperti berikut.

```
Activities  Terminal  Fri 10 Apr, 09:03
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help
Port /dev/ttyS0, 11:35:05

Press CTRL-A Z for help on special keys

5  (xinu)  Fri 10 Apr 11:57:09 EDT 2026

Found Intel 82545EM Ethernet NIC
MAC address is 08:00:27:9e:f4:08
   4447448 bytes of free memory.  Free list:
           [0x0015E320 to 0x0009FFF7]
           [0x00100000 to 0x005FBFFF]
    115180 bytes of Xinu code.
           [0x00100000 to 0x0011C1EB]
    148456 bytes of data.
           [0x0011FEE0 to 0x001442C7]

Obtained IP address  10.0.3.15   (0x0a00030f)
A
A
A
A
A
A
A
A
A
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:0 | ttyS0
```

Mengapa kode tersebut hanya menghasilkan output 'A' secara berulang?
Kode tersebut hanya menghasilkan output 'A' yang berulang karena program tersebut dijalankan secara sekuensial/berurutan. Ketika process main dimulai, maka fungsi sndA() akan dipanggil. Fungsi sndA() tersebut merupakan *infinity loop* (while(1)/while true), di mana ia akan berulang kali mecetak karakter 'A' dan tidur selama 1 detik. Karena eksekusi program harus menyelesaikan sndA() terlebih dahulu sebelum beralih ke instruksi berikutnya di main, dan karena sndA() tidak pernah selesai, maka program akan terjebak di fungsi itu selamanya dan fungsi sndB() tidak akan terpanggil dan tidak akan di eksekusi.

B. Konkuren Program

Berikut adalah contoh konkuren pada sistem operasi Xinu.
Pertama, kita kembali masuk ke direktori system dan berikan perintah **gedit main.c.**



Isi file main.c sesuai modu praktikum seperti di bawah ini. Jangan lupa untuk klik save.

```
#include <xinu.h>
void sndA(void);
void sndB(void);

process main(void)
{
        resume(create(sndA, 1024, 20, "process A", 0));
        resume(create(sndB, 1024, 20, "process B", 0));
        return OK;
}

void sndA(void)
{
        while(1){
        printf("A\n");
        sleepms(1000);
        }

}

void sndB(void)
{
        while(1){
        printf("B\n");
        sleepms(1000);
        }

}
```

Selanjutnya, kembali ke direktori xinu, lalu masuk ke direktori compile. Lalu, berikan perintah **make clean** dan **make**.

```
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
```

Jika berhasil, maka program akan berjalan dan outputnya akan seperti berikut.

```
Xinu for Vbox -- version #116  (xinu)  Fri 10 Apr 12:25:32 EDT 2026

Found Intel 82545EM Ethernet NIC
MAC address is 08:00:27:9e:f4:08
   4447448 bytes of free memory.  Free list:
           [0x0015E320 to 0x0009FFF7]
           [0x00100000 to 0x005FBFFF]
    115308 bytes of Xinu code.
           [0x00100000 to 0x0011C26B]
    148456 bytes of data.
           [0x0011FEE0 to 0x001442C7]

Obtained IP address  10.0.3.15   (0x0a00030f)
A
B
AB

A
B
A
B
A
B
A
B
A
B
```

Terakhir, untuk mengembalikan main.c seperti awal, kita bisa remove main.c pada direktori system (**rm main.c**). Buka direktori xinu, lalu pindahkan main.c pada xinu ke system (**mv main.c home/xinu/xinu/system**).

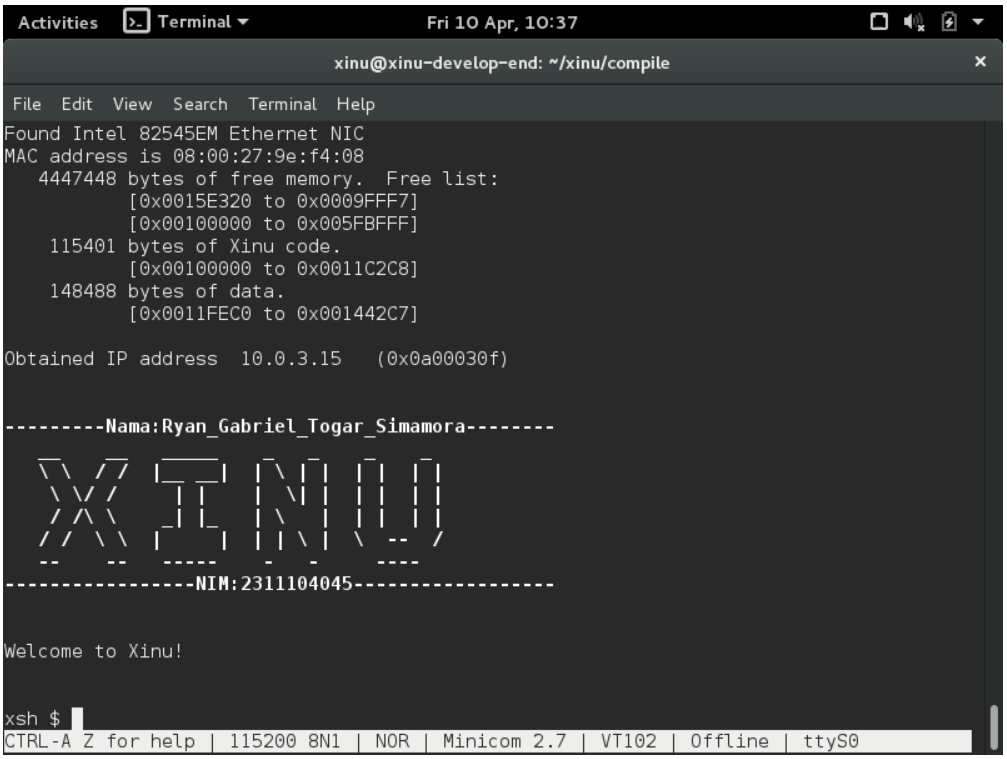
```
xinu@xinu-develop-end:~/xinu$ ls
compile  COPYRIGHT    device   lib     net    system
config   DESCRIPTION  include  main.c  shell
xinu@xinu-develop-end:~/xinu$ mv main.c /home/xinu/xinu/system
xinu@xinu-develop-end:~/xinu$ ls
compile  config  COPYRIGHT  DESCRIPTION  device  include  lib  net  shell  system
xinu@xinu-develop-end:~/xinu$
```

Jangan Lupa ke direktori compile lalu make clean dan make

```
Activities  Terminal  Fri 10 Apr, 10:25
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
```

Tambahin nama nim sesuai yang kemarin

```
Activities  Terminal  Fri 10 Apr, 10:28
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help
Found Intel 82545EM Ethernet NIC
MAC address is 08:00:27:9e:f4:08
   4447448 bytes of free memory.  Free list:
           [0x0015E320 to 0x0009FFF7]
           [0x00100000 to 0x005FBFFF]
    115337 bytes of Xinu code.
           [0x00100000 to 0x0011C288]
    148488 bytes of data.
           [0x0011FEC0 to 0x001442C7]

Obtained IP address  10.0.3.15   (0x0a00030f)


----------------------------------------
   __   __   _____   _   _   _   _
   \ \  / /  |__ __|  | \ |  |  |  |
    \ \/ /     | |    |  \|  |  |  |
    / /\ \    _| |_   | \   |  |  |
   / /  \ \  |     |  | | \ |  \  --  /
   --    --   -----    -   -    ----
----------------------------------------


Welcome to Xinu!


xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:0 | ttyS0
```



2.2 Jurnal Praktikum

1. [10 Poin] Selain hardware (memory), batasan maksimal proses dapat ditentukan dengan
secara software. Pada Linux maksimal proses adalah 4194303 proses (64 bit) dan 32767
proses (32 bit) dapat dilihat melalui perintah $cat /proc/sys/kernel/pid_max
Carilah pada source code Xinu yang memberi batasan mengenai banyaknya proses yang
bisa dibuat! Berapa maksimal proses dalam Xinu? Ubah menjadi maksimal 150 proses!

Jawab :

Maksimal proses pada xinu adalah 8 namun akan saya ubah jadi 150

```
/* process.h - isbadpid */

/* Maximum number of processes in the system */

#ifndef NPROC
#define NPROC           8
#endif
```

Ganti angka 8 jadi 150 dan save

```
/* process.h - isbadpid */

/* Maximum number of processes in the system */

#ifndef NPROC
#define NPROC           150
#endif
```

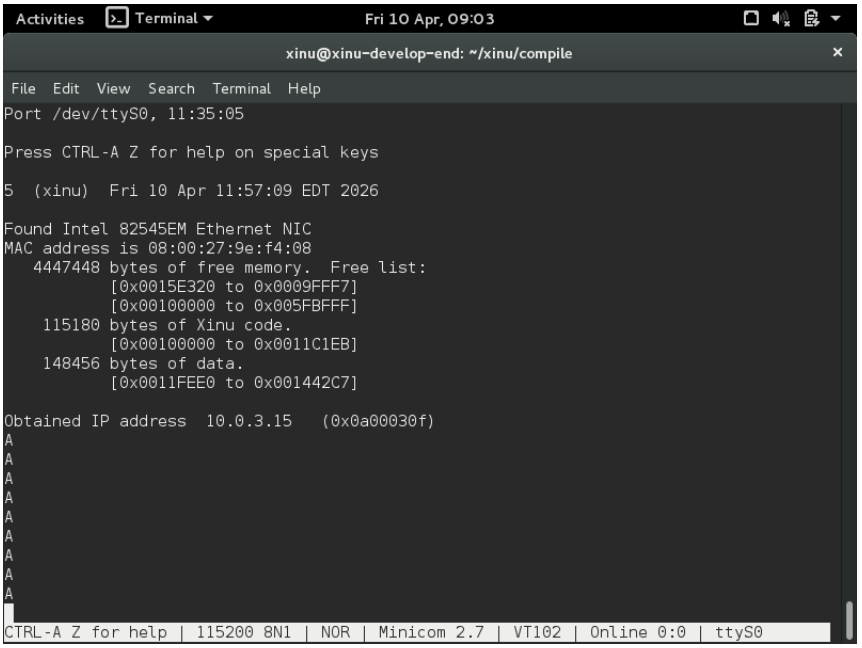
Setelah itu jangan lupa ke direktori compile dan jalankan perintah make clean dan make

```
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
```

2. [20 Poin] Jalankan kode main_ex1!

   Jawab :

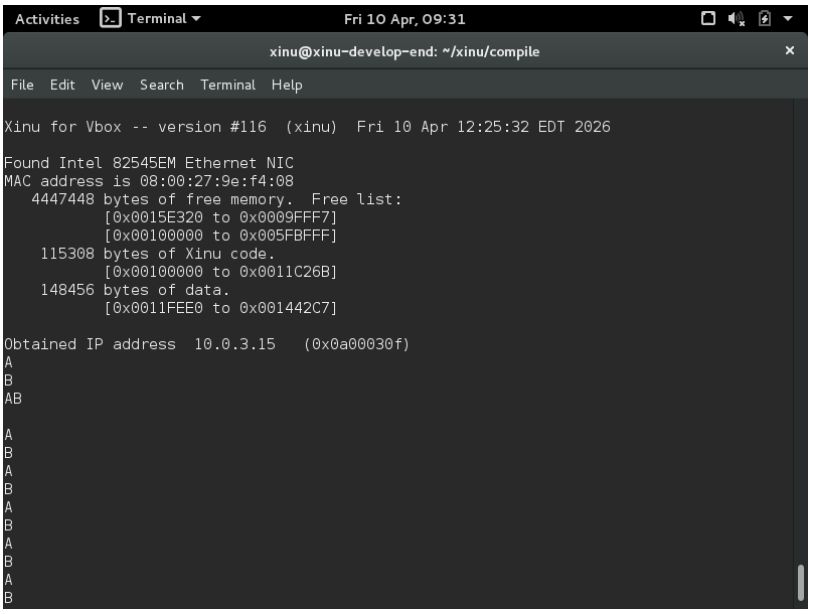
   Untuk file main_ex1 itu udh saya praktekkan di guided bagian **Sekuensial Program**



3. [20 Poin] Jalankan kode main_ex2!

   Jawab :

   Untuk file main_ex2 itu udh saya praktekkan di guided bagian **Konkuren Program**



4. [20 Poin] Jalankan kode main_ex3!

Jawab :

```
/* main_ex3.c - Prosedur dan Konsumer */
/* Nama : Ryan Gabriel Togar Simamora */
/* Nim : 2311104045 */

#include <xinu.h>
void sndChar(char);

process main(void)
{
        resume(create(sndChar, 1024, 20, "Send A", 1,"A"));
        resume(create(sndChar, 1024, 20, "Send B", 1,"B"));

}

void sndChar(char c)
{
        while(1){
        printf("%c \n", c);
        sleepms(1000);
        }
}
```

Setelah itu jangan lupa ke direktori compile dan jalankan perintah make clean dan make

```
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
```

Hasil Output

```
Xinu for Vbox -- version #98  (xinu)  Fri 10 Apr 16:52:24 EDT 2026

Found Intel 82545EM Ethernet NIC
MAC address is 08:00:27:9e:f4:08
   4447448 bytes of free memory.  Free list:
           [0x0015E320 to 0x0009FFF7]
           [0x00100000 to 0x005FBFFF]
    115401 bytes of Xinu code.
           [0x00100000 to 0x0011C2C8]
    148456 bytes of data.
           [0x0011FEE0 to 0x001442C7]

Obtained IP address  10.0.3.15   (0x0a00030f)
t
}
t
}
}
t
t
}
t
}
t}

CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:0 | ttyS0
```

5. [30 Poin] Buatlah 2 proses produser dan konsumer. Produser memproduksi angka integer dari 1-1000. Konsumer mengkonsumsi integer yang diproduksi oleh produser dan menampilkannya! (Gunakan variabel global bertipe int32 bernama n yang digunakan
secara bersama oleh kedua proses)

```
int32 n = 0; //global variabel
void produser(void){
int32 i;
for (i=1; i<=1000; i++){
n++;
}
}
void konsumer(void){
int32 i;
for (i=1; i<=1000; i++){
 printf("Nilai dari n adalah %d\n",n);
}
}
```

Hasil dari program ini cukup mengejutkan (tidak akan sesuai dengan intuisi awal). Jelaskan mengapa hasilnya seperti itu

Jawab :

Edit main.c dengan perintah gedit dan ganti isinya

```
/* main_ex4.c - Prosedur dan Konsumer */
/* Nama : Ryan Gabriel Togar Simamora */
/* Nim : 2311104045 */

#include <xinu.h>

int32 n = 0; // global variabel

void produser(void) {
        int32 i;
        for (i = 1; i <= 1000; i++) {
                n++;
        }
}


void konsumer(void) {
        int32 i;
        for (i = 1; i <= 1000; i++) {
                printf("Nilai dari n adalah %d\n", n);
        }
}

process main(void) {
        pid32 pid_prod, pid_kons;

        pid_prod = create(produser, 1024, 20, "produser", 0);
        pid_kons = create(konsumer, 1024, 20, "konsumer", 0);
```

Jangan lupa ke direktori compile dan jalankan perintah make clean dan make

```
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
```

Hasil Output

```
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
```

Penjelasan :
Output yang dihasilkan tidak akan berurutan dari 1 sampai 1000 melainkan looping nilai dari n adalah 1000
Hasil yang terjadi adalah konsumer hanya mencetak nilai n = 1000 sebanyak 1000 kali. Hal ini terjadi karena tidak ada sinkronisasi antara produser dan konsumer. Scheduler Xinu menjalankan produser hingga selesai terlebih dahulu (n mencapai 1000), baru kemudian memberikan giliran CPU ke konsumer. Akibatnya, konsumer selalu membaca nilai n yang sudah final yaitu 1000, bukan nilai yang sedang bertambah. Inilah yang disebut **race condition** yang dimana hasil program bergantung pada urutan eksekusi proses, bukan logika yang diinginkan programmer.

Solusi dari saya adalah menggunakan **semaphore** untuk memastikan produser dan konsumer bergantian secara teratur (sinkronisasi).

BAB III
PENUTUP

Kesimpulan

Kesimpulannya, perbedaan utama antara program sekuensial dan konkuren terletak pada cara sistem operasi mengelola urutan pekerjaan**.**

Sekuensial

- Cara Kerja: Instruksi dieksekusi satu per satu secara ketat dari awal hingga akhir.
- Kelemahan: Jika ada satu tugas yang membutuhkan waktu lama (misalnya, *loop* tak terbatas), tugas-tugas berikutnya tidak akan pernah dieksekusi. Ini membuang waktu CPU jika tugas yang berjalan harus menunggu (seperti menunggu I/O).
- Contoh: Dalam kode, pemanggilan sndA() yang memiliki *loop* tak terbatas membuat sndB() tidak pernah dipanggil.

Konkuren

- Cara Kerja: Tugas-tugas dipecah menjadi proses-proses terpisah (dibuat menggunakan create() dan resume()) dan dijalankan secara *interleaving* oleh *scheduler* OS.
- Keunggulan: Menciptakan ilusi keserentakan (terutama pada CPU tunggal) dan memungkinkan sistem untuk menggunakan waktu CPU secara efisien. Ketika satu proses beristirahat (misalnya, menggunakan sleepms()), OS segera beralih untuk menjalankan proses lain.
- Contoh: Dalam kode yang dimodifikasi, sndA dan sndB berjalan bergantian, menghasilkan *output* 'A' dan 'B' secara selang-seling, karena satu proses memberi jalan kepada proses lain saat ia tidur.